

Error Handling middleware all concept explain

I'll explain the **Error Handling Middleware** concept comprehensively, from basics to advanced patterns.

What is Error Handling Middleware?

Error handling middleware is a special type of middleware that catches and processes errors that occur during request processing. It acts as a centralized mechanism to handle application errors uniformly.

Key Characteristics

1. Signature Difference

Unlike regular middleware, error middleware has **4 parameters**:

```
javascript
```

```
// Regular middleware
```

```
app.use((req, res, next) => {})
```

```
// Error handling middleware
```

```
app.use((err, req, res, next) => {})
```

2. Placement Matters

```
javascript
```

```
app.use(regularMiddleware)
```

```
app.use(routeHandler) // Routes
```

```
app.use(errorMiddleware) // MUST be Last
```

How It Works - Visual Flow

text

Request → Route Handler → (Error Occurs)



next(error) → Error Middleware



Format Error + Send Response

Implementation Examples

Basic Example

javascript

```
const express = require('express');
const app = express();

// Regular route that might throw error
app.get('/user/:id', (req, res, next) => {
  const userId = req.params.id;

  if (userId === '0') {
    const error = new Error('Invalid user ID');
    error.status = 400;
    return next(error); // Pass error to middleware
  }

  res.json({ id: userId, name: 'John' });
});

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500);
  res.json({
    error: {
      message: err.message,
      status: err.status || 500
    }
  });
});
```

Multiple Error Handlers

javascript

```
// Not Found handler (404)
app.use((req, res, next) => {
  res.status(404).json({ error: 'Route not found' });
});

// Generic error handler (catches everything else)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
});
```

Error Propagation Patterns

1. Synchronous Errors

javascript

```
app.get('/sync-error', (req, res) => {
  throw new Error('Boom!'); // Automatically caught
  // Express will pass this to error middleware
});
```

2. Asynchronous Errors

javascript

```
// Without try-catch (WRONG)
app.get('/async-error', async (req, res, next) => {
  const data = await db.query(); // May reject
  res.json(data);
  // ❌ This won't be caught
});

// With next() (CORRECT)
app.get('/async-error', async (req, res, next) => {
  try {
```

```

    const data = await db.query();
    res.json(data);
  } catch (error) {
    next(error); //  Pass to error middleware
  }
});

// Using wrapper (ADVANCED)
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

app.get('/async-error', asyncHandler(async (req, res) => {
  const data = await db.query(); // Errors automatically passed to next()
  res.json(data);
})));

```

Advanced Patterns

1. Custom Error Classes

javascript

```

class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}

class ValidationError extends AppError {
  constructor(message) {
    super(message, 400);
    this.name = 'ValidationError';
  }
}

// Usage
app.post('/user', (req, res, next) => {
  if (!req.body.email) {
    next(new ValidationError('Email is required'));
  }
}

```



```
    }  
  });  
  
  // Specialized error handler  
  app.use((err, req, res, next) => {  
    if (err instanceof ValidationError) {  
      return res.status(400).json({ error: err.message });  
    }  
    next(err); // Pass to next error handler  
  });
```

2. Environment-Specific Responses

javascript

```
app.use((err, req, res, next) => {  
  err.statusCode = err.statusCode || 500;  
  
  if (process.env.NODE_ENV === 'development') {  
    res.status(err.statusCode).json({  
      error: err.message,  
      stack: err.stack,  
      status: err.statusCode  
    });  
  } else {  
    // Production - hide technical details  
    res.status(err.statusCode).json({  
      error: err.isOperational ? err.message : 'Internal Server Error'  
    });  
  }  
});
```

3. Logging Integration

javascript

```
const winston = require('winston');  
const logger = winston.createLogger({  
  transports: [new winston.transports.File({ filename: 'error.log' })]  
});
```

```
app.use((err, req, res, next) => {
  // Log error details
  logger.error({
    message: err.message,
    stack: err.stack,
    url: req.url,
    method: req.method,
    ip: req.ip,
    userId: req.user?.id
  });

  // Send response
  res.status(err.status || 500).json({
    error: 'Internal Server Error',
    requestId: req.id // For tracking in logs
  });
});
```

4. Database Error Handling

javascript

```
app.use((err, req, res, next) => {
  // MongoDB duplicate key error
  if (err.code === 11000) {
    return res.status(409).json({
      error: 'Duplicate entry',
      field: Object.keys(err.keyPattern)[0]
    });
  }

  // PostgreSQL unique violation
  if (err.code === '23505') {
    return res.status(409).json({
      error: 'Resource already exists'
    });
  }

  // Validation errors
  if (err.name === 'ValidationError') {
    return res.status(400).json({
      error: err.message
    });
  }
});
```

```
}  
  
    next(err);  
  });
```

Best Practices

✓ DO

javascript

```
// 1. Always have a final error handler  
app.use((err, req, res, next) => {  
  res.status(500).json({ error: 'Something broke!' });  
});  
  
// 2. Pass errors to next()  
app.get('/route', (req, res, next) => {  
  someAsyncOperation().catch(next);  
});  
  
// 3. Set appropriate status codes  
app.use((err, req, res, next) => {  
  const status = err.status || 500;  
  res.status(status).json({ error: err.message });  
});  
  
// 4. Handle 404 explicitly  
app.use((req, res) => {  
  res.status(404).json({ error: 'Not found' });  
});
```

✗ DON'T

javascript

```
// 1. Don't ignore errors  
app.get('/route', (req, res) => {  
  asyncOperation(); // No error handling  
});
```

```
// 2. Don't throw errors in async handlers without try-catch
app.get('/route', async (req, res) => {
  throw new Error('This crashes the server!');
});

// 3. Don't place error middleware before routes
app.use(errorHandler); // Wrong placement!
app.get('/route', handler);

// 4. Don't forget to return after sending response
app.use((err, req, res, next) => {
  res.json({ error: err.message });
  next(err); // ❌ This would call next error handler
});
```

Framework-Specific Patterns

Express.js

javascript

```
const errorHandler = (err, req, res, next) => {
  // Implementation
};
```

Koa.js (Uses async/await)

javascript

```
app.use(async (ctx, next) => {
  try {
    await next();
  } catch (err) {
    ctx.status = err.status || 500;
    ctx.body = { error: err.message };
  }
});
```

NestJS (Class-based)

typescript

```
@Catch(HttpException)
export class HttpExceptionHandler implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse();
    response.status(exception.getStatus()).json({
      error: exception.message
    });
  }
}
```

Complete Production Example

javascript

```
const express = require('express');
const helmet = require('helmet');

const app = express();
app.use(helmet());

// Global error handler factory
const createErrorHandler = (options = {}) => {
  return (err, req, res, next) => {
    // Set default status
    err.status = err.status || 500;

    // Log error
    if (options.logger) {
      options.logger.error({
        message: err.message,
        stack: err.stack,
        url: req.url,
        method: req.method,
        ip: req.ip,
        status: err.status
      });
    }
  };
}
```

```
// Send response
res.status(err.status).json({
  error: {
    message: options.hideDetails && err.status === 500
      ? 'Internal Server Error'
      : err.message,
    status: err.status,
    ...(options.showStack && { stack: err.stack })
  }
});
};

// Usage
app.use(createErrorHandler({
  logger: console,
  hideDetails: process.env.NODE_ENV === 'production',
  showStack: process.env.NODE_ENV === 'development'
}));
```

Summary

Error handling middleware provides:

- **Centralized error management**
- **Consistent error responses**
- **Separation of concerns** (business logic vs error handling)
- **Graceful degradation**
- **Better debugging** with proper logging

Remember: **Always place error middleware at the end, always pass errors to next(), and handle both sync and async errors properly.**

custom error class

Custom Error Classes - Complete Guide

Custom error classes extend JavaScript's built-in `Error` class to create **domain-specific errors** with additional properties and behaviors.

Why Custom Error Classes?

Problems with Generic Errors

javascript

```
// ❌ Generic error - Lacks context
throw new Error('Something went wrong');
// No status code, no error type, no additional data
```

Benefits of Custom Errors

javascript

```
// ✅ Custom error - rich context
throw new ValidationError('Invalid email format', {
  field: 'email',
  value: 'invalid',
  suggestion: 'user@example.com'
});
// Has status code, error type, actionable metadata
```

Basic Custom Error Class

javascript

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message); // Call parent Error constructor
    this.statusCode = statusCode;
    this.isOperational = true; // Distinguish from programming errors

    // Capture stack trace (excludes constructor from trace)
    Error.captureStackTrace(this, this.constructor);
  }
}
```

```
// Usage
throw new AppError('User not found', 404);
```

Complete Hierarchy of Custom Errors

javascript

```
// Base Error Class
class AppError extends Error {
  constructor(message, statusCode, details = null) {
    super(message);
    this.name = this.constructor.name; // Sets class name as error name
    this.statusCode = statusCode;
    this.details = details;
    this.isOperational = true;
    this.timestamp = new Date().toISOString();

    Error.captureStackTrace(this, this.constructor);
  }
}

// 4xx Client Errors
class ValidationError extends AppError {
  constructor(message, field = null, value = null) {
    super(message, 400);
    this.field = field;
    this.invalidValue = value;
  }
}

class AuthenticationError extends AppError {
  constructor(message = 'Authentication required') {
    super(message, 401);
  }
}

class AuthorizationError extends AppError {
  constructor(message = 'Insufficient permissions') {
    super(message, 403);
  }
}
```



```
class NotFoundError extends AppError {
  constructor(resourceType, identifier) {
    super(`${resourceType} not found: ${identifier}`, 404);
    this.resourceType = resourceType;
    this.identifier = identifier;
  }
}

class ConflictError extends AppError {
  constructor(resource, existingField) {
    super(`${resource} already exists`, 409);
    this.existingField = existingField;
  }
}

class RateLimitError extends AppError {
  constructor(retryAfterSeconds) {
    super('Too many requests', 429);
    this.retryAfter = retryAfterSeconds;
  }
}

// 5xx Server Errors
class DatabaseError extends AppError {
  constructor(message, originalError = null) {
    super(message, 500);
    this.originalError = originalError;
  }
}

class ExternalServiceError extends AppError {
  constructor(serviceName, originalError = null) {
    super(`${serviceName} service unavailable`, 503);
    this.serviceName = serviceName;
    this.originalError = originalError;
  }
}
```

Advanced Custom Error Implementation

1. With Validation Framework Integration

javascript

```
class ValidationError extends AppError {
  constructor(errors) {
    super('Validation failed', 400);
    this.errors = this.formatErrors(errors);
  }

  formatErrors(errors) {
    if (Array.isArray(errors)) {
      return errors.map(err => ({
        field: err.path,
        message: err.message,
        value: err.value
      }));
    }

    if (typeof errors === 'object') {
      return Object.entries(errors).map(([field, message]) => ({
        field,
        message,
        value: null
      }));
    }

    return [{ message: errors }];
  }

  // Helper method
  hasField(fieldName) {
    return this.errors.some(err => err.field === fieldName);
  }
}

// Usage with Joi
const Joi = require('joi');

app.post('/user', (req, res, next) => {
  const schema = Joi.object({
    email: Joi.string().email().required(),
    age: Joi.number().min(18).required()
  });

  const { error } = schema.validate(req.body);
```

```
    if (error) {  
      return next(new ValidationError(error.details));  
    }  
    // Process request...  
  });
```

2. Database Error Wrapper

javascript

```
class DatabaseError extends AppError {  
  constructor(operation, originalError) {  
    super(`Database ${operation} failed`, 500);  
    this.operation = operation;  
    this.code = originalError.code;  
    this.detail = this.parseDatabaseError(originalError);  
  }  
  
  parseDatabaseError(error) {  
    // Handle specific database errors  
    switch(error.code) {  
      case '23505': // PostgreSQL unique violation  
        return 'Duplicate entry';  
      case '23503': // Foreign key violation  
        return 'Referenced record not found';  
      case '42P01': // Table doesn't exist  
        return 'Invalid table reference';  
      default:  
        return error.message;  
    }  
  }  
  
  // Business Logic specific methods  
  isDuplicateKeyError() {  
    return this.code === '23505';  
  }  
}  
  
// Usage  
try {  
  await db.query('INSERT INTO users VALUES ($1, $2)', [email, name]);  
} catch(error) {
```

```
    throw new DatabaseError('insert', error);  
  }  
}
```

3. Chainable/Builder Pattern

javascript

```
class AppError extends Error {  
  constructor(message) {  
    super(message);  
    this.metadata = {};  
    this.statusCode = 500;  
  }  
  
  status(code) {  
    this.statusCode = code;  
    return this;  
  }  
  
  withDetail(key, value) {  
    this.metadata[key] = value;  
    return this;  
  }  
  
  operational() {  
    this.isOperational = true;  
    return this;  
  }  
  
  cause(originalError) {  
    this.cause = originalError;  
    return this;  
  }  
}  
  
// Usage - Fluent interface  
throw new AppError('Payment failed')  
  .status(402)  
  .withDetail('transactionId', 'txn_123')  
  .withDetail('amount', 99.99)  
  .operational()  
  .cause(originalError);
```

Serialization Methods

javascript

```
class AppError extends Error {
  constructor(message, statusCode, metadata = {}) {
    super(message);
    this.statusCode = statusCode;
    this.metadata = metadata;
  }

  // For JSON.stringify()
  toJSON() {
    return {
      name: this.name,
      message: this.message,
      statusCode: this.statusCode,
      metadata: this.metadata,
      timestamp: this.timestamp,
      // Exclude stack in production
      ...(process.env.NODE_ENV !== 'production' && { stack: this.stack })
    };
  }

  // For Logging
  toLog() {
    return {
      error: this.name,
      message: this.message,
      status: this.statusCode,
      metadata: this.metadata,
      stack: this.stack,
      timestamp: new Date().toISOString()
    };
  }

  // For client response (sanitized)
  toResponse() {
    const response = {
      error: this.name,
      message: this.message,
      statusCode: this.statusCode
    };
  }
}
```

```

    // Only include details in development or if explicitly requested
    if (process.env.NODE_ENV === 'development') {
        response.details = this.metadata;
    }

    return response;
}
}
}

```

Real-World Implementation

javascript

```

// errors/CustomErrors.js
class APIError extends Error {
    constructor(message, statusCode, errorCode, details = null) {
        super(message);
        this.name = this.constructor.name;
        this.statusCode = statusCode;
        this.errorCode = errorCode; // Custom error codes for frontend
        this.details = details;
        this.isOperational = true;

        Error.captureStackTrace(this, this.constructor);
    }
}

class UserError extends APIError {
    static notFound(userId) {
        return new UserError(
            `User with ID ${userId} not found`,
            404,
            'USER_NOT_FOUND',
            { userId }
        );
    }

    static duplicateEmail(email) {
        return new UserError(
            `Email ${email} already registered`,
            409,
            'EMAIL_TAKEN',
            { email }
        );
    }
}

```

```
    );
  }

  static invalidCredentials() {
    return new UserError(
      'Invalid email or password',
      401,
      'INVALID_CREDENTIALS'
    );
  }

  static inactive() {
    return new UserError(
      'Account is deactivated',
      403,
      'ACCOUNT_INACTIVE'
    );
  }
}

class PaymentError extends APIError {
  static insufficientFunds(required, available) {
    return new PaymentError(
      `Insufficient funds: Required ${required}, Available ${available}`,
      402,
      'INSUFFICIENT_FUNDS',
      { required, available }
    );
  }

  static paymentFailed(transactionId, reason) {
    return new PaymentError(
      `Payment failed: ${reason}`,
      402,
      'PAYMENT_FAILED',
      { transactionId, reason }
    );
  }
}

// Export all errors
module.exports = {
  APIError,
  UserError,
```

```
PaymentError
```

```
};
```

Error Handler with Custom Errors

```
javascript
```

```
// middleware/errorHandler.js
```

```
const { APIError } = require('../errors/CustomErrors');
```

```
const errorHandler = (err, req, res, next) => {
```

```
  // Default values
```

```
  let statusCode = 500;
```

```
  let response = {
```

```
    success: false,
```

```
    error: 'Internal Server Error',
```

```
    message: 'Something went wrong'
```

```
  };
```

```
  // Handle custom API errors
```

```
  if (err instanceof APIError) {
```

```
    statusCode = err.statusCode;
```

```
    response = {
```

```
      success: false,
```

```
      error: err.name,
```

```
      message: err.message,
```

```
      code: err.errorCode
```

```
    };
```

```
    // Add details in development
```

```
    if (process.env.NODE_ENV === 'development' && err.details) {
```

```
      response.details = err.details;
```

```
    }
```

```
  }
```

```
  // Handle Joi validation errors
```

```
  else if (err.name === 'ValidationError') {
```

```
    statusCode = 400;
```

```
    response = {
```

```
      success: false,
```

```
      error: 'ValidationError',
```

```
      message: 'Validation failed',
```

```
      details: err.details || err.message
```



```

    };
  }

  // Handle JWT errors
  else if (err.name === 'JsonWebTokenError') {
    statusCode = 401;
    response = {
      success: false,
      error: 'AuthError',
      message: 'Invalid token'
    };
  }

  // Log unexpected errors
  if (statusCode === 500) {
    console.error('Unhandled error:', err);
  }

  res.status(statusCode).json(response);
};

module.exports = errorHandler;

```

TypeScript Version

typescript

```

// errors.ts
interface ErrorDetails {
  [key: string]: any;
}

class AppError extends Error {
  public readonly statusCode: number;
  public readonly isOperational: boolean;
  public readonly details?: ErrorDetails;
  public readonly timestamp: Date;

  constructor(
    message: string,
    statusCode: number = 500,
    details?: ErrorDetails
  ) {

```

```

        super(message);
        this.name = this.constructor.name;
        this.statusCode = statusCode;
        this.details = details;
        this.isOperational = true;
        this.timestamp = new Date();

        Error.captureStackTrace(this, this.constructor);
    }
}

class NotFoundError extends AppError {
    constructor(resource: string, id: string | number) {
        super(`${resource} with id ${id} not found`, 404, { resource, id });
    }
}

class ValidationError extends AppError {
    constructor(errors: Record<string, string>) {
        super('Validation failed', 400, { errors });
    }
}

// Generic type-safe error
class BusinessError<T> extends Record<string, any>> extends AppError {
    constructor(
        message: string,
        statusCode: number,
        public readonly context: T
    ) {
        super(message, statusCode, context);
    }
}

// Usage
type OrderContext = {
    orderId: string;
    userId: string;
    amount: number;
};

throw new BusinessError<OrderContext>(
    'Order already processed',
    409,

```

```
{ orderId: '123', userId: '456', amount: 99.99 }  
);
```

Best Practices

✓ DO

javascript

```
// 1. Extend built-in Error properly  
class MyError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = this.constructor.name;  
    Error.captureStackTrace(this, this.constructor);  
  }  
}  
  
// 2. Use meaningful error names  
class UserNotFoundError extends Error {} // Clear and specific  
class DBConnectionError extends Error {}  
  
// 3. Include actionable data  
throw new ValidationError('Email invalid', {  
  field: 'email',  
  received: 'user@',  
  expected: 'valid email format'  
});  
  
// 4. Add helper methods for common checks  
if (error.isNotFound()) { /* handle */ }  
if (error.isRetryable()) { /* retry logic */ }  
  
// 5. Create factory methods for common scenarios  
UserError.fromDatabase(error);  
ValidationError.fromJoi(error);
```

✗ DON'T

javascript

```

// 1. Don't forget to call super()
class BadError extends Error {
  constructor() {
    // super() missing - message won't be set
  }
}

// 2. Don't override name manually (unless needed)
class MyError extends Error {
  constructor() {
    super();
    this.name = 'Error'; // ❌ Destroys error type info
  }
}

// 3. Don't use generic errors for specific cases
throw new Error('User not found'); // ❌ Prefer UserNotFoundError

// 4. Don't expose internal details
throw new Error(`Query failed: ${sqlQuery}`); // ❌ Exposes internals

// 5. Don't lose original error
try {
  await db.query();
} catch(err) {
  throw new DatabaseError(); // ❌ Original error lost
  // ✅ throw new DatabaseError('Query failed', err);
}

```

Testing Custom Errors

javascript

```

describe('Custom Errors', () => {
  it('should create validation error with details', () => {
    const error = new ValidationError('Email required', 'email', '');

    expect(error).toBeInstanceOf(ValidationError);
    expect(error).toBeInstanceOf(AppError);
    expect(error.statusCode).toBe(400);
    expect(error.field).toBe('email');
    expect(error.message).toBe('Email required');
  });
}

```

```
});

it('should serialize correctly for response', () => {
  const error = new UserNotFoundError('user_123');
  const response = error.toResponse();

  expect(response).toEqual({
    error: 'UserNotFoundError',
    message: 'User user_123 not found',
    statusCode: 404
  });
});

it('should have proper stack trace', () => {
  const error = new AppError('Test error', 500);
  expect(error.stack).toBeDefined();
  expect(error.stack).toContain('Test error');
});
});
```

Summary

Custom error classes provide:

- **Type safety** - Differentiate between error types
- **Rich context** - Include status codes, error codes, metadata
- **Better error handling** - Specific handling per error type
- **Improved debugging** - Clear error hierarchy and stack traces
- **API consistency** - Uniform error response structure
- **Operational clarity** - Distinguish between operational and programming errors

Use them to make your error handling **predictable, maintainable, and developer-friendly!**

Default status and message all concept easy

Default Status and Messages - Complete Easy Guide

The Core Concept

When handling errors, you need **default values** when specific information isn't available. Think of it like a backup plan:

javascript

```
// If we don't know what went wrong...
status = error.status || 500
message = error.message || "Something went wrong"

// || means: "use the left side, or if that's missing, use the right side"
```

Simple Real-World Analogy

text

Restaurant Order:

- If customer says "Make it spicy" → Use spicy (specific)
- If customer says nothing → Use mild (default)

Same with errors:

- If error has status 404 → Use 404 (specific)
- If error has no status → Use 500 (default)

The Most Common Pattern

javascript

```
// Basic error handler with defaults
app.use((err, req, res, next) => {
  const status = err.status || 500;           // Default: 500
  const message = err.message || 'Server Error'; // Default: Server Error

  res.status(status).json({ error: message });
});
```

Why Do We Need Defaults?

Problem Without Defaults

javascript

```
// ❌ No defaults - CRASHES!
app.use((err, req, res, next) => {
  res.status(err.status).json({ error: err.message });
  // If err.status is undefined → res.status(undefined) → ERROR!
});
```

Solution With Defaults

javascript

```
// ✅ With defaults - SAFE!
app.use((err, req, res, next) => {
  const status = err.status || 500; // Always a number
  const message = err.message || 'Unknown error'; // Always a string

  res.status(status).json({ error: message }); // Always works!
});
```

Default Status Codes Explained

HTTP Status Code Groups

javascript

```
// 2xx: Success (rarely used for errors)
// 4xx: Client Errors (you did something wrong)
// 5xx: Server Errors (we did something wrong)

const DEFAULT_STATUS = {
  CLIENT_ERROR: 400, // Bad Request
  SERVER_ERROR: 500, // Internal Server Error
  NOT_FOUND: 404, // Resource doesn't exist
  UNAUTHORIZED: 401, // Not Logged in
  FORBIDDEN: 403 // Not allowed
};
```

When to Use Which Default

javascript

```
// 💡 DIFFERENT DEFAULTS FOR DIFFERENT SCENARIOS

// For validation errors
const status = err.status || 400; // Default: Bad Request

// For database errors
const status = err.status || 500; // Default: Server Error

// For missing data
const status = err.status || 404; // Default: Not Found

// For auth errors
const status = err.status || 401; // Default: Unauthorized
```

Complete Easy Examples

Example 1: Simple Default Values

javascript

```
// Without custom error
app.get('/user', (req, res, next) => {
  try {
    throw new Error(); // No status, no message!
  } catch(error) {
    next(error); // Pass empty error to handler
  }
});

// Error handler provides defaults
app.use((err, req, res, next) => {
  // Default values kick in
  const status = err.status || 500;
  const message = err.message || 'Operation failed';

  res.status(status).json({
    success: false,
    error: message
  });
});
```



```
    });  
  });  
  
  // Response: { success: false, error: "Operation failed" }  
  // Status: 500
```

Example 2: Partial Error Information

javascript

```
app.get('/product/:id', (req, res, next) => {  
  const product = findProduct(req.params.id);  
  
  if (!product) {  
    const error = new Error();  
    error.status = 404; // 🖱️ We set status, but NO message  
    return next(error);  
  }  
});  
  
app.use((err, req, res, next) => {  
  // status: 404 (from error)  
  // message: "Not Found" (default from ||)  
  const status = err.status || 500;  
  const message = err.message || 'Not Found';  
  
  res.status(status).json({ error: message });  
});  
// Response: { error: "Not Found" } with status 404
```

Example 3: Complete Default Object Pattern

javascript

```
// Create a defaults object  
const DEFAULTS = {  
  status: 500,  
  message: 'Internal Server Error',  
  code: 'UNKNOWN_ERROR'  
};
```

```
app.use((err, req, res, next) => {  
  // Spread defaults, then override with actual error values  
  const response = {  
    status: DEFAULTS.status,  
    message: DEFAULTS.message,  
    code: DEFAULTS.code,  
    ...err // This overwrites defaults with real error data  
  };  
  
  res.status(response.status).json({  
    error: response.message,  
    code: response.code  
  });  
});
```

Common Default Patterns

Pattern 1: The Ternary Check

javascript

```
// Long way  
let status;  
if (err.status) {  
  status = err.status;  
} else {  
  status = 500;  
}  
  
// Short way (|| operator)  
const status = err.status || 500;  
  
// Even shorter (?? operator - checks only null/undefined)  
const status = err.status ?? 500;
```

Pattern 2: Multiple Defaults

javascript

```
app.use((err, req, res, next) => {  
  // Status defaults chain  
  const status = err.status  
    || err.statusCode  
    || err.code  
    || 500;  
  
  // Message defaults chain  
  const message = err.message  
    || err.msg  
    || err.error  
    || 'Unknown Error';  
  
  res.status(status).json({ error: message });  
});
```

Pattern 3: Conditional Defaults

javascript

```
app.use((err, req, res, next) => {  
  let status;  
  let message;  
  
  // Different defaults based on error type  
  if (err.name === 'ValidationError') {  
    status = err.status || 400;  
    message = err.message || 'Invalid input';  
  }  
  else if (err.name === 'AuthError') {  
    status = err.status || 401;  
    message = err.message || 'Authentication required';  
  }  
  else {  
    status = err.status || 500;  
    message = err.message || 'Server error';  
  }  
  
  res.status(status).json({ error: message });  
});
```

Helper Functions for Defaults

Defaults Factory

javascript

```
// Create reusable default handler
function createErrorHandler(defaultStatus = 500, defaultMessage = 'Error') {
  return (err, req, res, next) => {
    const status = err.status || defaultStatus;
    const message = err.message || defaultMessage;

    res.status(status).json({ error: message });
  };
}

// Use different defaults for different routes
app.use('/api', createErrorHandler(400, 'API Error'));
app.use('/admin', createErrorHandler(500, 'Admin Error'));
app.use(createErrorHandler()); // Uses default 500
```

Defaults with Environment

javascript

```
app.use((err, req, res, next) => {
  // Different defaults for different environments
  const isDev = process.env.NODE_ENV === 'development';

  const status = err.status || 500;
  const message = err.message || (isDev ? 'Unknown error' : 'Server Error');

  const response = {
    error: message
  };

  // Add stack trace only in development
  if (isDev && err.stack) {
    response.stack = err.stack || 'No stack trace';
  }
});
```

```
res.status(status).json(response);  
});
```

Real-World Examples

Example: E-Commerce API

javascript

```
// Default messages catalog  
const DEFAULT_MESSAGES = {  
  400: 'Invalid request data',  
  401: 'Please login to continue',  
  403: 'You don\'t have permission',  
  404: 'Resource not found',  
  409: 'Conflict with existing data',  
  429: 'Too many requests, please try later',  
  500: 'Something went wrong on our end',  
  503: 'Service temporarily unavailable'  
};  
  
app.use((err, req, res, next) => {  
  // Get status with default 500  
  const status = err.status || 500;  
  
  // Get custom message or use default from catalog  
  const message = err.message || DEFAULT_MESSAGES[status] || 'Unknown error';  
  
  res.status(status).json({  
    success: false,  
    error: message,  
    statusCode: status  
  });  
});  
  
// Usage examples:  
throw { status: 404 };  
// Response: { success: false, error: "Resource not found", statusCode: 404 }  
  
throw { status: 401, message: "Session expired" };  
// Response: { success: false, error: "Session expired", statusCode: 401 }
```

```
throw {};  
// Response: { success: false, error: "Something went wrong on our end", statusCode: 500  
}
```

Example: Input Validation

javascript

```
function validateUserInput(data) {  
  const errors = [];  
  
  if (!data.email) {  
    errors.push({  
      field: 'email',  
      message: 'Email is required'  
    });  
  }  
  
  if (!data.age) {  
    errors.push({  
      field: 'age',  
      message: 'Age is required'  
    });  
  } else if (data.age < 18) {  
    errors.push({  
      field: 'age',  
      message: 'Must be 18 or older'  
    });  
  }  
  
  if (errors.length > 0) {  
    const error = new Error();  
    error.status = 400;  
    error.message = errors[0].message; // First error as main message  
    error.details = errors;           // All errors as details  
    throw error;  
  }  
}  
  
app.post('/user', (req, res, next) => {  
  try {  
    validateUserInput(req.body);  
    res.json({ success: true });  
  }  
});
```

```
    } catch(err) {
      next(err);
    }
  });

  app.use((err, req, res, next) => {
    const status = err.status || 500;
    const message = err.message || 'Validation failed';

    res.status(status).json({
      error: message,
      details: err.details || [] // Empty array default
    });
  });
```

Quick Reference Chart

Scenario	Default Status	Default Message
No error info provided	500	"Internal Server Error"
Only status provided	As provided	"An error occurred"
Only message provided	500	As provided
Validation error	400	"Invalid input"
Not found error	404	"Resource not found"
Auth error	401	"Authentication required"
Database error	500	"Database operation failed"

Common Mistakes & Solutions

✖ Mistake 1: Wrong Order

javascript

```
// Wrong - will always use default
const status = 500 || err.status; // Always 500

// Correct - default only if no error status
const status = err.status || 500; // Uses error status if exists
```

✗ Mistake 2: Using `||` for Zero Values

```
javascript

// Problem: 0 is falsy, so default will be used
const status = err.status || 500;
// If err.status = 0 → status becomes 500 (WRONG!)

// Solution: Use ?? (nullish coalescing)
const status = err.status ?? 500;
// If err.status = 0 → status stays 0 (CORRECT!)
```

✗ Mistake 3: Forgetting Default for Response

```
javascript

// Wrong - may send undefined
res.json(err.message); // If message undefined → sends nothing

// Correct - always have default
res.json({ error: err.message || 'Error' });
```

Best Practices Summary

```
javascript

// 1. Always provide defaults
const status = err.status ?? 500;
const message = err.message ?? 'Unknown error';

// 2. Group defaults in one place
const ERROR_DEFAULTS = {
  status: 500,
```



```

    message: 'Something went wrong'
  };

// 3. Use descriptive default messages
const message = err.message || {
  400: 'Bad request',
  404: 'Not found',
  500: 'Server error'
}[err.status] || 'Error occurred';

// 4. Log when using defaults (helps debugging)
if (!err.status) {
  console.warn('Error missing status, using 500');
}

// 5. Create default error objects
function createDefaultError() {
  return {
    status: 500,
    message: 'Unknown error',
    timestamp: new Date()
  };
}

```

Simple Memory Trick

text

Think of `||` as a safety net:

```

error.status || 500
    ↑         ↑
  Try this  Fallback to this

```

If left side exists → use it

If left side missing → use right side

Final Simple Example

javascript

```
// The simplest error handler you'll ever need
app.use((err, req, res, next) => {
  res.status(err.status || 500)
    .json({ error: err.message || 'Oops! Something broke!' });
});

// That's it! Just these 3 lines handle ALL errors
// With sensible defaults for everything
```

Remember: Defaults are your backup dancers - they step in when the star (specific error info) isn't available! 🎯

activity

Error Handling Middleware - Complete Learning Activity

Here's a **hands-on activity** to help you master error handling middleware concepts through practice.

Activity 1: Build from Scratch (Beginner)

Step 1: Setup Basic Express Server

```
javascript

// server.js
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json());

// Your code will go here

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Step 2: Create Routes with Different Error Scenarios

javascript

```
// Task 1: Create a route that throws a synchronous error
app.get('/sync-error', (req, res) => {
  // TODO: Throw an error "Sync Boom!"
});

// Task 2: Create a route that passes error to next()
app.get('/async-error', (req, res, next) => {
  // TODO: Create error "Async Error" and pass to next()
});

// Task 3: Create a route that handles missing users
app.get('/user/:id', (req, res, next) => {
  const users = {
    '1': { name: 'John' },
    '2': { name: 'Jane' }
  };

  // TODO: If user not found, create error with status 404
  // TODO: If user found, return user data
});

// Task 4: Create a route that requires age validation
app.post('/register', (req, res, next) => {
  const { name, age } = req.body;

  // TODO: If age < 18, create error "Must be 18+"
  // TODO: If name missing, create error "Name required"
  // TODO: If all good, return success message
});

// Task 5: Create error handling middleware (LAST!)
app.use((err, req, res, next) => {
  // TODO: Log error to console
  // TODO: Use defaults (status 500, message "Server Error")
  // TODO: Send JSON response with error details
});

// Task 6: Handle 404 for unmatched routes
// TODO: Create middleware for routes that don't exist
```

Activity 2: Fill in the Blanks

Complete the missing parts:

javascript

```
// Error handler template
app.use((err, req, res, next) => {
  // 1. Get status with default
  const status = err._____ || 500;

  // 2. Get message with default
  const message = err._____ || '_____';

  // 3. Log error
  console._____(`[ERROR] ${status}: ${message}`);

  // 4. Send response
  res._____(status)._____(
    {
      success: false,
      error: _____,
      timestamp: new Date()._____()
    }
  );
});

// Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    _____(message); // Call parent constructor
    this.statusCode = _____;
    this.name = '_____';
    Error.captureStackTrace(this, this._____);
  }
}
```

Activity 3: Debug the Code

Find and fix the 5 bugs in this error handler:

javascript

```
// BUGGY CODE - Find the issues!
app.use((err, req, res, next) => {
  const status = err.status;
  const message = err.message;

  res.status(status).json({
    error: message
  });

  next();
});

app.get('/buggy', (req, res) => {
  throw 'String error';
});

app.get('/missing', (req, res, next) => {
  const data = null;
  data.name = 'test'; // This will crash
  res.json(data);
});
```

List the bugs you found:

- 1.
- 2.
- 3.
- 4.
- 5.

Activity 4: Build a Custom Error Class System

Task: Create a complete error hierarchy

javascript

```
// TODO: Create base AppError class
class AppError extends Error {
  constructor(message, statusCode, details = null) {
    // Your code here
  }
}
```

```

    }

    // TODO: Create specific error types
    class ValidationError extends AppError {
      constructor(message, field) {
        // Your code here
      }
    }

    class NotFoundError extends AppError {
      constructor(resource, id) {
        // Your code here
      }
    }

    class AuthError extends AppError {
      constructor(message) {
        // Your code here
      }
    }

    // TODO: Create routes that use these errors
    app.get('/product/:id', (req, res, next) => {
      // Throw NotFoundError if product doesn't exist
    });

    app.post('/validate', (req, res, next) => {
      // Throw ValidationError if email invalid
    });

    app.get('/admin', (req, res, next) => {
      // Throw AuthError if not admin
    });

```

Activity 5: Error Handler with Multiple Features

Build a comprehensive error handler:

```

javascript

// TODO: Create an enhanced error handler with:
// 1. Different handling for development vs production
// 2. Logging to file

```

```
// 3. Rate limiting for errors
// 4. Email alerts for critical errors

const errorHandler = (err, req, res, next) => {
  const isDev = process.env.NODE_ENV === 'development';

  // TODO: Get status with default
  // TODO: Get message with default

  // TODO: Log error with different levels
  // - Console.error for all
  // - File logging for 500 errors
  // - Email for critical errors

  // TODO: Create response object
  const response = {
    // Add fields based on environment
  };

  // TODO: Send response
};

// TODO: Add 404 handler before this
```

Activity 6: Practical Challenge - Build an API with Error Handling

Create a **Todo API** with proper error handling:

```
javascript

let todos = [
  { id: 1, task: 'Learn error handling', completed: false },
  { id: 2, task: 'Build API', completed: false }
];

// TODO: GET /todos - Get all todos
app.get('/todos', (req, res, next) => {
  // Should always succeed
});

// TODO: GET /todos/:id - Get single todo
```

```
app.get('/todos/:id', (req, res, next) => {
  // If id not found → 404 error
  // If id invalid → 400 error
});

// TODO: POST /todos - Create todo
app.post('/todos', (req, res, next) => {
  const { task } = req.body;
  // If task missing → 400 error
  // If task empty → 400 error
});

// TODO: PUT /todos/:id - Update todo
app.put('/todos/:id', (req, res, next) => {
  const { completed } = req.body;
  // If todo not found → 404
  // If completed not boolean → 400
});

// TODO: DELETE /todos/:id - Delete todo
app.delete('/todos/:id', (req, res, next) => {
  // If todo not found → 404
  // Success → 204 No Content
});

// TODO: Add error handling middleware
```

Activity 7: Testing Your Error Handler

Create test scenarios:

```
javascript

// Test script - run with node or in browser console
const testErrors = async () => {
  const tests = [
    { url: '/sync-error', expectedStatus: 500 },
    { url: '/user/999', expectedStatus: 404 },
    { url: '/invalid-route', expectedStatus: 404 },
    {
      url: '/register',
      method: 'POST',
      body: { age: 16 },
    }
  ]
}
```



```

        expectedStatus: 400
      },
      {
        url: '/register',
        method: 'POST',
        body: { name: 'John', age: 25 },
        expectedStatus: 200
      }
    ];

    for (const test of tests) {
      try {
        const response = await fetch(`http://localhost:3000${test.url}`, {
          method: test.method || 'GET',
          headers: { 'Content-Type': 'application/json' },
          body: test.body ? JSON.stringify(test.body) : undefined
        });

        console.log(`✓ ${test.url}: ${response.status} (Expected: ${test.expectedStat
us})`);
      } catch (error) {
        console.log(`X ${test.url}: Failed - ${error.message}`);
      }
    }
  };

  // Run tests
  testErrors();

```

Activity 8: Real-World Scenario - Payment System

Build error handling for a payment system:

```

javascript

// Payment error types
class PaymentError extends Error {
  constructor(message, statusCode, transactionId = null) {
    super(message);
    this.statusCode = statusCode;
    this.transactionId = transactionId;
  }
}

```

```

class InsufficientFundsError extends PaymentError {
  constructor(required, available, transactionId) {
    super(`Insufficient funds. Need ${required}, have ${available}`, 402, transactionId);
    this.required = required;
    this.available = available;
  }
}

class CardDeclinedError extends PaymentError {
  constructor(reason, transactionId) {
    super(`Card declined: ${reason}`, 402, transactionId);
    this.reason = reason;
  }
}

// TODO: Create payment processing route
app.post('/payment', async (req, res, next) => {
  const { amount, cardNumber } = req.body;
  const transactionId = Math.random().toString(36);

  // TODO: Add validation
  // TODO: Process payment with error handling
  // TODO: Handle specific payment errors
});

// TODO: Create specialized error handler for payments

```

Activity 9: Error Handler with Metrics

Add monitoring and metrics:

```

javascript

// Error metrics tracking
const errorMetrics = {
  total: 0,
  byStatus: {},
  byType: {},
  lastError: null
};

```

```
app.use((err, req, res, next) => {
  // TODO: Track total errors
  // TODO: Track by status code
  // TODO: Track by error type
  // TODO: Record last error time

  // TODO: Log metrics every 10 errors
  // TODO: Send alert if error rate > 10 per minute
});

// TODO: Create endpoint to view metrics
app.get('/admin/metrics', (req, res) => {
  // Only allow in development
  // Return error metrics
});
```

Activity 10: Complete Mini-Project

Build a **Library Management System** with comprehensive error handling:

javascript

```
// Requirements:
// 1. CRUD operations for Books
// 2. CRUD operations for Members
// 3. Borrow/Return system
// 4. Error handling for all scenarios

// Data structure
let books = [];
let members = [];
let borrowRecords = [];

// Custom errors needed:
// - BookNotFoundError
// - MemberNotFoundError
// - BookNotAvailableError
// - MaxBooksReachedError
// - LateReturnError
// - DuplicateISBNError

// TODO: Implement all routes with proper error handling
// TODO: Create middleware for logging
```

```
// TODO: Create validation middleware  
// TODO: Create global error handler
```

Answer Key for Activity 2

javascript

```
// Fill in the blanks answers:  
app.use((err, req, res, next) => {  
  const status = err.status || 500;  
  const message = err.message || 'Internal Server Error';  
  
  console.error(`[ERROR] ${status}: ${message}`);  
  
  res.status(status).json({  
    success: false,  
    error: message,  
    timestamp: new Date().toISOString()  
  });  
});  
  
class AppError extends Error {  
  constructor(message, statusCode) {  
    super(message);  
    this.statusCode = statusCode;  
    this.name = 'AppError';  
    Error.captureStackTrace(this, this.constructor);  
  }  
}
```

Self-Assessment Checklist

After completing activities, check your understanding:

- I can create basic error handling middleware
- I understand why defaults (||) are important
- I can create custom error classes
- I know the difference between sync and async error handling
- I can differentiate 4xx vs 5xx errors

- I can implement validation error handling
- I know how to handle 404 routes
- I can create environment-specific error responses
- I understand error propagation with next()
- I can build a complete error handling system

Challenge Yourself

Bonus Task: Create an error handler that:

1. Supports multiple languages (error messages in English/Spanish)
2. Has a "retry" suggestion for certain errors
3. Includes a unique error ID for tracking
4. Sends Slack notification for critical errors
5. Writes to a database for analytics

javascript

```
// Sample output of advanced error handler
{
  "success": false,
  "error": {
    "id": "err_abc123",
    "message": "Database connection failed",
    "code": "DB_CONN_ERROR",
    "language": "en",
    "retry": true,
    "retryAfter": 5,
    "suggestion": "Check your internet connection",
    "helpUrl": "https://docs.api.com/errors/DB_CONN_ERROR"
  },
  "timestamp": "2024-01-15T10:30:00Z"
}
```

Quick Practice Commands

bash

```
# Test your error handler with curl
curl http://localhost:3000/sync-error
curl http://localhost:3000/user/999
curl -X POST http://localhost:3000/register -H "Content-Type: application/json" -d '{"age":16}'
curl http://localhost:3000/not-exists
```

Remember: The best way to learn is to break things intentionally and see how your error handler responds! 🚀

Handling Async errors

Handling Async Errors - Complete Easy Guide

The Core Problem

Async errors **don't automatically trigger** Express error handlers. You must manually pass them to `next()`.

javascript

```
// ❌ THIS DOESN'T WORK - Async error crashes server
app.get('/user', async (req, res) => {
  const user = await db.find(); // If this fails → CRASH!
  res.json(user);
});
```

```
// ✅ THIS WORKS - Pass error to next()
app.get('/user', async (req, res, next) => {
  try {
    const user = await db.find();
    res.json(user);
  } catch(error) {
    next(error); // ✅ Triggers error handler
  }
});
```

Why Async Errors Are Different

javascript

```
// Synchronous error - Express catches automatically
app.get('/sync', (req, res) => {
  throw new Error('Boom!'); // ✅ Auto-caught by Express
});

// Asynchronous error - Express can't catch
app.get('/async', async (req, res) => {
  throw new Error('Boom!'); // ❌ CRASH! Express can't catch
  // Because the function returns a Promise, not the error
});
```

Solution 1: Try-Catch (Beginner Friendly)

Basic Pattern

javascript

```
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    res.json(user);
  } catch (error) {
    next(error); // Send to error handler
  }
});
```

Multiple Async Operations

javascript

```
app.post('/order', async (req, res, next) => {
  try {
    // Multiple async operations
    const user = await User.findById(req.body.userId);
    const product = await Product.findById(req.body.productId);
    const payment = await processPayment(req.body.amount);

    res.json({ user, product, payment });
  }
});
```

```
    } catch (error) {  
      next(error); // Catches ANY error from above  
    }  
  });
```

With Custom Error Types

javascript

```
app.get('/product/:id', async (req, res, next) => {  
  try {  
    const product = await Product.findById(req.params.id);  
  
    if (!product) {  
      // Create custom error  
      const error = new Error('Product not found');  
      error.status = 404;  
      throw error; // Throw to try-catch  
    }  
  
    res.json(product);  
  } catch (error) {  
    next(error);  
  }  
});
```

Solution 2: Async Handler Wrapper (Intermediate)

Create a Wrapper Function

javascript

```
// Wrapper that catches async errors automatically  
const asyncHandler = (fn) => {  
  return (req, res, next) => {  
    fn(req, res, next).catch(next); // Catch and pass to next()  
  };  
};  
  
// Usage - No more try-catch needed!
```



```
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
}));

app.post('/order', asyncHandler(async (req, res) => {
  const order = await createOrder(req.body);
  res.json(order);
}));
```

Multiple Routes with Wrapper

javascript

```
// ALL routes now have async error handling
const routes = [
  { method: 'get', path: '/users', handler: getUsers },
  { method: 'get', path: '/users/:id', handler: getUser },
  { method: 'post', path: '/users', handler: createUser },
  { method: 'put', path: '/users/:id', handler: updateUser },
  { method: 'delete', path: '/users/:id', handler: deleteUser }
];

routes.forEach(route => {
  app[route.method](route.path, asyncHandler(route.handler));
});
```

Solution 3: Express 5 (Automatic)

Express 5 automatically catches async errors:

javascript

```
// Express 5 - No need for try-catch or wrapper!
const express = require('express'); // Version 5.x
const app = express();

app.get('/user/:id', async (req, res) => {
  const user = await User.findById(req.params.id);
  // Errors automatically go to error handler!
```

```
    res.json(user);  
  });
```

Note: Express 5 is still beta. Most use Express 4 with wrappers.

Real-World Examples

Example 1: Database Operations

javascript

```
// User API with database errors  
app.get('/users', async (req, res, next) => {  
  try {  
    const users = await db.query('SELECT * FROM users');  
    res.json(users);  
  } catch (error) {  
    // Enhance error with context  
    error.message = `Database error: ${error.message}`;  
    error.status = 500;  
    next(error);  
  }  
});  
  
app.get('/users/:id', async (req, res, next) => {  
  try {  
    const user = await db.query('SELECT * FROM users WHERE id = $1', [req.params.id]);  
  
    if (user.rows.length === 0) {  
      const error = new Error('User not found');  
      error.status = 404;  
      throw error;  
    }  
  
    res.json(user.rows[0]);  
  } catch (error) {  
    next(error);  
  }  
});
```

Example 2: External API Calls

javascript

```
const axios = require('axios');

app.get('/weather/:city', async (req, res, next) => {
  try {
    const response = await axios.get(`https://api.weather.com/${req.params.city}`);
    res.json(response.data);
  } catch (error) {
    // Handle different API error scenarios
    if (error.response) {
      // API responded with error status
      error.status = error.response.status;
      error.message = `Weather API error: ${error.response.data.message}`;
    } else if (error.request) {
      // No response received
      error.status = 503;
      error.message = 'Weather service unavailable';
    } else {
      // Other errors
      error.status = 500;
      error.message = 'Failed to fetch weather data';
    }
    next(error);
  }
});
```

Example 3: File Operations

javascript

```
const fs = require('fs').promises;
const path = require('path');

app.get('/files/:filename', async (req, res, next) => {
  try {
    const filePath = path.join(__dirname, 'uploads', req.params.filename);
    const data = await fs.readFile(filePath, 'utf8');
    res.send(data);
  } catch (error) {
    if (error.code === 'ENOENT') {

```

```
        error.message = 'File not found';
        error.status = 404;
    } else {
        error.message = 'Error reading file';
        error.status = 500;
    }
    next(error);
}
});
```

Advanced Patterns

Pattern 1: Async Class Methods

javascript

```
class UserController {
  async getUser(req, res, next) {
    try {
      const user = await User.findById(req.params.id);
      res.json(user);
    } catch (error) {
      next(error);
    }
  }

  async createUser(req, res, next) {
    try {
      const user = await User.create(req.body);
      res.status(201).json(user);
    } catch (error) {
      next(error);
    }
  }
}

const userController = new UserController();
app.get('/users/:id', (req, res, next) => userController.getUser(req, res, next));
app.post('/users', (req, res, next) => userController.createUser(req, res, next));
```

Pattern 2: Promise Version (without async/await)

javascript

```
// Using promises directly
app.get('/user/:id', (req, res, next) => {
  User.findById(req.params.id)
    .then(user => res.json(user))
    .catch(next); // Pass error to next()
});

// Multiple promises
app.get('/dashboard', (req, res, next) => {
  Promise.all([
    User.getStats(),
    Order.getRecent(),
    Product.getPopular()
  ])
    .then(([stats, orders, products]) => {
      res.json({ stats, orders, products });
    })
    .catch(next);
});
```

Pattern 3: Error Wrapper for Router

javascript

```
// Wrapper for entire router
const asyncRouter = (router) => {
  const methods = ['get', 'post', 'put', 'delete', 'patch'];

  methods.forEach(method => {
    const originalMethod = router[method];
    router[method] = function(path, ...handlers) {
      // Wrap each handler with async error handling
      const wrappedHandlers = handlers.map(handler => {
        return async (req, res, next) => {
          try {
            await handler(req, res, next);
          } catch (error) {
            next(error);
          }
        }
      });
    };
  });
};
```

```

        };
    });
    return originalMethod.call(this, path, ...wrappedHandlers);
  });
});

return router;
};

// Usage
const router = express.Router();
asyncRouter(router);

router.get('/user/:id', async (req, res) => {
  const user = await User.findById(req.params.id); // Auto-caught!
  res.json(user);
});

```

Testing Async Error Handling

javascript

```

// Test file (test.js)
const request = require('supertest');
const app = require('./app');

describe('Async Error Handling', () => {
  test('should handle database error', async () => {
    const response = await request(app)
      .get('/user/invalid')
      .expect(500);

    expect(response.body.error).toBeDefined();
  });

  test('should handle not found error', async () => {
    const response = await request(app)
      .get('/user/99999')
      .expect(404);

    expect(response.body.error).toBe('User not found');
  });
});

```

```

test('should handle validation error', async () => {
  const response = await request(app)
    .post('/user')
    .send({ name: '' })
    .expect(400);

  expect(response.body.error).toContain('Name required');
});
});

```

Common Async Error Scenarios

Scenario 1: Missing await

javascript

```

// ❌ BUG - Missing await
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = User.findById(req.params.id); // Missing await!
    res.json(user); // Sends Promise, not user data
  } catch (error) {
    next(error);
  }
});

```

```

// ✅ CORRECT
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    res.json(user);
  } catch (error) {
    next(error);
  }
});

```

Scenario 2: Not catching Promise rejections

javascript

```
// ❌ BUG - No catch
app.get('/data', (req, res, next) => {
  fetchData() // Returns Promise but no catch
    .then(data => res.json(data));
  // If fetchData rejects → Unhandled Promise Rejection
});

// ✅ CORRECT - With catch
app.get('/data', (req, res, next) => {
  fetchData()
    .then(data => res.json(data))
    .catch(next);
});
```

Scenario 3: Async in middleware

javascript

```
// ❌ BUG - Async middleware without error handling
app.use(async (req, res, next) => {
  await session.validate(req); // If fails → crash
  next();
});

// ✅ CORRECT
app.use(async (req, res, next) => {
  try {
    await session.validate(req);
    next();
  } catch (error) {
    next(error);
  }
});
```

Complete Production Example

javascript

```
// asyncHandler.js - Reusable wrapper
const asyncHandler = (fn) => {
  return (req, res, next) => {
```



```
    Promise.resolve(fn(req, res, next)).catch(next);
  };
};

// Custom error types
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Database service
class DatabaseService {
  static async findUser(id) {
    // Simulate database error
    if (id === 'error') {
      throw new AppError('Database connection failed', 503);
    }
    if (id === '999') {
      return null;
    }
    return { id, name: 'John Doe' };
  }
}

// Routes with asyncHandler
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await DatabaseService.findUser(req.params.id);

  if (!user) {
    throw new AppError('User not found', 404);
  }

  res.json({
    success: true,
    data: user
  });
}));

app.post('/user', asyncHandler(async (req, res) => {
  const { name, email } = req.body;

  if (!name || !email) {
```

```
        throw new AppError('Name and email required', 400);
    }

    const user = await DatabaseService.createUser({ name, email });
    res.status(201).json({
        success: true,
        data: user
    });
}));

// Error handling middleware
app.use((err, req, res, next) => {
    const status = err.statusCode || 500;
    const message = err.message || 'Internal Server Error';

    console.error(`[ERROR] ${status}: ${message}`);

    res.status(status).json({
        success: false,
        error: message,
        ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
    });
});

// 404 handler for unmatched routes
app.use('*', (req, res) => {
    res.status(404).json({
        success: false,
        error: 'Route not found'
    });
});
});
```

Quick Reference Card

Scenario	Solution
Single async operation	try-catch + next(error)
Multiple async operations	Single try-catch around all
All routes	Use asyncHandler wrapper

Scenario	Solution
Express 5	Automatic handling
Promises	<code>.catch(next)</code>
Class methods	Wrap in try-catch
Middleware	Try-catch + next

Best Practices Summary

javascript

//  DO THIS

```
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await db.find(req.params.id);
  if (!user) throw new AppError('Not found', 404);
  res.json(user);
}));
```

//  NOT THIS

```
app.get('/user/:id', async (req, res) => {
  const user = await db.find(req.params.id); // No error handling!
  res.json(user);
});
```

//  DO THIS (without wrapper)

```
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await db.find(req.params.id);
    res.json(user);
  } catch (error) {
    next(error);
  }
});
```

Remember: Always wrap async operations in error handling. Express can't catch async errors automatically (unless using Express 5)! 

Using Try-Catch for Async Error Handling - Complete Easy Guide

The Basics: Why Try-Catch?

Try-catch is the **most straightforward** way to handle async errors in Express.

javascript

```
// The pattern
app.get('/route', async (req, res, next) => {
  try {
    // Your async code here
    const data = await someAsyncOperation();
    res.json(data);
  } catch (error) {
    next(error); // Pass to Express error handler
  }
});
```

Simple Examples to Start

Example 1: Basic Database Query

javascript

```
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    res.json(user);
  } catch (error) {
    next(error); // Database error goes here
  }
});
```

Example 2: Multiple Async Operations

javascript

```
app.post('/order', async (req, res, next) => {
  try {
    // ALL these could fail, but one try-catch handles everything
    const user = await User.findById(req.body.userId);
    const product = await Product.findById(req.body.productId);
    const payment = await processPayment(req.body.amount);
    const email = await sendConfirmationEmail(user.email);

    res.json({ success: true, orderId: payment.orderId });
  } catch (error) {
    next(error); // Catches ANY error from above
  }
});
```

Real-World Examples

Example 1: User Registration with Validation

javascript

```
app.post('/register', async (req, res, next) => {
  try {
    const { email, password, name } = req.body;

    // Validation
    if (!email || !password) {
      const error = new Error('Email and password required');
      error.status = 400;
      throw error; // Throw to catch block
    }

    // Check if user exists
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      const error = new Error('Email already registered');
      error.status = 409;
      throw error;
    }

    // Create user
```

```

const user = await User.create({ email, password, name });

// Send welcome email (don't fail registration if email fails)
try {
  await sendWelcomeEmail(email);
} catch (emailError) {
  console.error('Email failed:', emailError);
  // Don't throw - registration still successful
}

res.status(201).json({
  success: true,
  userId: user.id,
  message: 'Registration successful'
});

} catch (error) {
  next(error);
}
});

```

Example 2: File Upload with Processing

javascript

```

const multer = require('multer');
const upload = multer({ dest: 'uploads/' });

app.post('/upload', upload.single('file'), async (req, res, next) => {
  try {
    if (!req.file) {
      const error = new Error('No file uploaded');
      error.status = 400;
      throw error;
    }

    // Process file
    const processedData = await processFile(req.file.path);

    // Save to database
    const savedData = await Database.save(processedData);

    // Clean up temp file

```

```

    await fs.unlink(req.file.path);

    res.json({
      success: true,
      data: savedData,
      message: 'File processed successfully'
    });

  } catch (error) {
    // Clean up temp file even on error
    if (req.file) {
      await fs.unlink(req.file.path).catch(console.error);
    }
    next(error);
  }
});

```

Example 3: External API Call with Retry Logic

javascript

```

app.get('/weather/:city', async (req, res, next) => {
  try {
    const { city } = req.params;
    let retries = 3;
    let lastError;

    // Retry logic inside try-catch
    for (let i = 0; i < retries; i++) {
      try {
        const response = await axios.get(`https://api.weather.com/${city}`);
        return res.json(response.data);
      } catch (error) {
        lastError = error;
        console.log(`Attempt ${i + 1} failed, retrying...`);
        await delay(1000 * (i + 1)); // Wait before retry
      }
    }

    // All retries failed
    throw lastError;

  } catch (error) {

```

```
// Enhance error with more info
error.status = error.response?.status || 503;
error.message = `Weather API error: ${error.message}`;
next(error);
}
});

function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

Different Ways to Throw Errors

Method 1: Throw Error Object

javascript

```
app.get('/data', async (req, res, next) => {
  try {
    throw new Error('Something went wrong');
  } catch (error) {
    next(error);
  }
});
```

Method 2: Throw with Status Code

javascript

```
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);

    if (!user) {
      const error = new Error('User not found');
      error.status = 404;
      throw error;
    }

    res.json(user);
  }
});
```



```
    } catch (error) {  
      next(error);  
    }  
  });  
});
```

Method 3: Create Custom Error Class

javascript

```
class AppError extends Error {  
  constructor(message, statusCode) {  
    super(message);  
    this.statusCode = statusCode;  
  }  
}  
  
app.get('/product/:id', async (req, res, next) => {  
  try {  
    const product = await Product.findById(req.params.id);  
  
    if (!product) {  
      throw new AppError('Product not found', 404);  
    }  
  
    if (product.price < 0) {  
      throw new AppError('Invalid product price', 400);  
    }  
  
    res.json(product);  
  } catch (error) {  
    next(error);  
  }  
});
```

Nested Try-Catch Patterns

Pattern 1: Inner Try-Catch for Non-Critical Operations

javascript

```
app.post('/order', async (req, res, next) => {
  try {
    // Critical operations
    const order = await createOrder(req.body);
    const payment = await processPayment(order.total);

    // Non-critical - won't fail the request
    try {
      await sendOrderConfirmation(order.userEmail);
      await updateInventory(order.items);
    } catch (backgroundError) {
      console.error('Background task failed:', backgroundError);
      // Log but don't rethrow
    }

    res.json({ success: true, order });

  } catch (error) {
    next(error); // Only critical errors reach here
  }
});
```

Pattern 2: Multiple Try-Catch for Different Operations

javascript

```
app.get('/dashboard', async (req, res, next) => {
  let userStats = null;
  let recentOrders = null;
  let recommendations = null;

  // Each operation can fail independently
  try {
    userStats = await getUserStats(req.userId);
  } catch (error) {
    console.error('Failed to get user stats:', error);
    userStats = { error: 'Unable to load stats' };
  }

  try {
    recentOrders = await getRecentOrders(req.userId);
  } catch (error) {
    console.error('Failed to get recent orders:', error);
    recentOrders = { error: 'Unable to load recent orders' };
  }

  // ... more try-catch blocks for other operations ...

  res.json({ userStats, recentOrders, recommendations });
});
```

```
    console.error('Failed to get orders:', error);
    recentOrders = [];
  }

  try {
    recommendations = await getRecommendations(req.userId);
  } catch (error) {
    console.error('Failed to get recommendations:', error);
    recommendations = [];
  }

  // Dashboard loads even if some parts fail
  res.json({
    userStats,
    recentOrders,
    recommendations
  });
});
```

Common Mistakes and Solutions

✗ Mistake 1: Forgetting to use `await`

javascript

```
// WRONG - Missing await
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = User.findById(req.params.id); // No await
    res.json(user); // Sends Promise instead of data
  } catch (error) {
    next(error); // Won't catch if User.findById rejects
  }
});
```

```
// CORRECT
app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    res.json(user);
  } catch (error) {
    next(error);
  }
});
```

```
}  
});
```

✗ Mistake 2: Not passing error to next()

javascript

```
// WRONG - Error not passed to Express  
app.get('/user/:id', async (req, res, next) => {  
  try {  
    const user = await User.findById(req.params.id);  
    res.json(user);  
  } catch (error) {  
    console.log(error); // Just logs, but no response sent!  
    // Request hangs!  
  }  
});  
  
// CORRECT  
app.get('/user/:id', async (req, res, next) => {  
  try {  
    const user = await User.findById(req.params.id);  
    res.json(user);  
  } catch (error) {  
    next(error); // Pass to Express error handler  
  }  
});
```

✗ Mistake 3: Throwing strings instead of Error objects

javascript

```
// WRONG - Throwing string  
app.get('/data', async (req, res, next) => {  
  try {  
    throw 'Something broke'; // String, not Error object  
  } catch (error) {  
    next(error); // error is a string, no stack trace  
  }  
});
```

```
// CORRECT - Throw Error object
app.get('/data', async (req, res, next) => {
  try {
    throw new Error('Something broke');
  } catch (error) {
    next(error);
  }
});
```

✗ Mistake 4: Try-catch without async

javascript

```
// WRONG - Try-catch not needed for sync code
app.get('/data', (req, res, next) => {
  try {
    const data = syncOperation(); // Synchronous
    res.json(data);
  } catch (error) {
    next(error);
  }
});

// CORRECT - Express handles sync errors automatically
app.get('/data', (req, res) => {
  const data = syncOperation();
  res.json(data);
});
```

Complete Working Example

javascript

```
const express = require('express');
const app = express();
app.use(express.json());

// Mock database
const database = {
  users: [{ id: 1, name: 'John', email: 'john@example.com' }],
```

```
    async findUserById(id) {
      await delay(100); // Simulate async
      const user = this.users.find(u => u.id === id);
      if (!user) {
        const error = new Error('User not found');
        error.code = 'USER_NOT_FOUND';
        error.status = 404;
        throw error;
      }
      return user;
    },

    async createUser(userData) {
      await delay(100);
      if (userData.email === 'exists@example.com') {
        const error = new Error('Email already exists');
        error.code = 'DUPLICATE_EMAIL';
        error.status = 409;
        throw error;
      }
      const user = { id: Date.now(), ...userData };
      this.users.push(user);
      return user;
    }
  };

  // Routes with try-catch
  app.get('/users/:id', async (req, res, next) => {
    try {
      const user = await database.findUserById(parseInt(req.params.id));
      res.json({ success: true, data: user });
    } catch (error) {
      next(error);
    }
  });

  app.post('/users', async (req, res, next) => {
    try {
      const { name, email } = req.body;

      // Validation
      if (!name || !email) {
        const error = new Error('Name and email are required');
        error.status = 400;
      }
    }
  });
```

```

        throw error;
    }

    if (!email.includes('@')) {
        const error = new Error('Invalid email format');
        error.status = 400;
        throw error;
    }

    // Create user
    const user = await database.createUser({ name, email });
    res.status(201).json({ success: true, data: user });

    } catch (error) {
        next(error);
    }
});

// Error handling middleware
app.use((err, req, res, next) => {
    const status = err.status || 500;
    const message = err.message || 'Internal Server Error';
    const code = err.code || 'UNKNOWN_ERROR';

    console.error(`[${status}] ${code}: ${message}`);

    res.status(status).json({
        success: false,
        error: {
            message,
            code,
            ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
        }
    });
});

// Helper function
function delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

// Start server
app.listen(3000, () => {

```

```
console.log('Server running on port 3000');  
});
```

Testing Your Try-Catch Handlers

javascript

```
// test.js - Test different scenarios  
async function testAPI() {  
  // Test 1: Valid user  
  await fetch('http://localhost:3000/users/1')  
    .then(r => r.json())  
    .then(data => console.log('Test 1:', data));  
  
  // Test 2: Invalid user (404)  
  await fetch('http://localhost:3000/users/999')  
    .then(r => r.json())  
    .then(data => console.log('Test 2:', data));  
  
  // Test 3: Create user with missing fields (400)  
  await fetch('http://localhost:3000/users', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ name: 'Jane' }) // Missing email  
  })  
    .then(r => r.json())  
    .then(data => console.log('Test 3:', data));  
  
  // Test 4: Duplicate email (409)  
  await fetch('http://localhost:3000/users', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      name: 'Jane',  
      email: 'exists@example.com'  
    })  
  })  
    .then(r => r.json())  
    .then(data => console.log('Test 4:', data));  
}  
  
testAPI();
```


Tips for Effective Try-Catch

javascript

```
// ✅ DO: Be specific with error messages
try {
  await processPayment();
} catch (error) {
  error.message = `Payment failed for user ${userId}: ${error.message}`;
  next(error);
}

// ✅ DO: Log errors with context
try {
  await saveToDatabase();
} catch (error) {
  console.error(`Database error for query: ${query}`, error);
  next(error);
}

// ✅ DO: Clean up resources in finally
let fileHandle;
try {
  fileHandle = await openFile();
  await processFile(fileHandle);
} catch (error) {
  next(error);
} finally {
  if (fileHandle) await closeFile(fileHandle);
}

// ❌ DON'T: Swallow errors
try {
  await riskyOperation();
} catch (error) {
  // Do nothing - bad!
}

// ❌ DON'T: Use try-catch for flow control
try {
  await userExists();
} catch (error) {

```

```
// Error means user doesn't exist - use if-else instead  
}
```

Quick Reference

Pattern	Code
Basic try-catch	<code>try { await operation() } catch(e) { next(e) }</code>
With status codes	<code>const err = new Error(); err.status = 404; throw err</code>
Custom error class	<code>throw new AppError('message', 404)</code>
Nested try-catch	Inner catch for non-critical failures
Finally block	Cleanup that runs always

Remember: Try-catch is your friend! It's simple, readable, and works everywhere. When in doubt, wrap your async code in try-catch and pass errors to `next()` ! 🎯

using `wrapAsync`

Using asyncHandler (wrapAsync) - Complete Easy Guide

What is asyncHandler?

`asyncHandler` (also called `wrapAsync`) is a **wrapper function** that automatically catches errors from async route handlers and passes them to Express error middleware.

javascript

```
// Without asyncHandler - need try-catch everywhere  
app.get('/user/:id', async (req, res, next) => {  
  try {  
    const user = await User.findById(req.params.id);  
    res.json(user);  
  } catch (error) {  
    next(error);  
  }  
})
```

```
});

// With asyncHandler - no try-catch needed!
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user); // Errors automatically go to next()
}));
```

The Basic asyncHandler Function

javascript

```
// The simplest version
function asyncHandler(fn) {
  return function(req, res, next) {
    fn(req, res, next).catch(next);
  };
}

// Same thing with arrow functions
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// Using it
app.get('/route', asyncHandler(async (req, res) => {
  const data = await db.query();
  res.json(data);
}));
```

Why Use asyncHandler?

Problem: Repetitive Try-Catch

javascript

```
// WITHOUT asyncHandler - So much repetition!
app.get('/users', async (req, res, next) => {
  try {
    const users = await User.find();
```

```
        res.json(users);
    } catch (err) { next(err); }
  });

app.get('/products', async (req, res, next) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (err) { next(err); }
});

app.post('/orders', async (req, res, next) => {
  try {
    const order = await Order.create(req.body);
    res.json(order);
  } catch (err) { next(err); }
});
```

Solution: Clean with asyncHandler

javascript

```
// WITH asyncHandler - Clean and DRY
app.get('/users', asyncHandler(async (req, res) => {
  const users = await User.find();
  res.json(users);
}));

app.get('/products', asyncHandler(async (req, res) => {
  const products = await Product.find();
  res.json(products);
}));

app.post('/orders', asyncHandler(async (req, res) => {
  const order = await Order.create(req.body);
  res.json(order);
}));
```

Complete Implementation Examples

Example 1: Basic Setup

javascript

```
const express = require('express');
const app = express();

// Define asyncHandler
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// Route using asyncHandler
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);

  if (!user) {
    throw new Error('User not found'); // Auto-caught!
  }

  res.json(user);
})));

app.listen(3000);
```

Example 2: Complete CRUD API

javascript

```
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// User CRUD operations
app.get('/users', asyncHandler(async (req, res) => {
  const users = await User.find();
  res.json({ success: true, data: users });
})));

app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);

  if (!user) {
```

```
    const error = new Error('User not found');
    error.status = 404;
    throw error;
  }

  res.json({ success: true, data: user });
}));

app.post('/users', asyncHandler(async (req, res) => {
  const { name, email } = req.body;

  if (!name || !email) {
    const error = new Error('Name and email required');
    error.status = 400;
    throw error;
  }

  const user = await User.create({ name, email });
  res.status(201).json({ success: true, data: user });
}));

app.put('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });

  if (!user) {
    const error = new Error('User not found');
    error.status = 404;
    throw error;
  }

  res.json({ success: true, data: user });
}));

app.delete('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findByIdAndDelete(req.params.id);

  if (!user) {
    const error = new Error('User not found');
    error.status = 404;
    throw error;
  }
}
```

```
res.json({ success: true, message: 'User deleted' }));  
}));
```

Advanced asyncHandler Patterns

Pattern 1: With Custom Error Class

javascript

```
class AppError extends Error {  
  constructor(message, statusCode) {  
    super(message);  
    this.statusCode = statusCode;  
  }  
}  
  
const asyncHandler = (fn) => (req, res, next) => {  
  Promise.resolve(fn(req, res, next)).catch(next);  
};  
  
app.get('/product/:id', asyncHandler(async (req, res) => {  
  const product = await Product.findById(req.params.id);  
  
  if (!product) {  
    throw new AppError('Product not found', 404);  
  }  
  
  if (product.quantity === 0) {  
    throw new AppError('Product out of stock', 400);  
  }  
  
  res.json(product);  
}));
```

Pattern 2: With Multiple Operations

javascript

```
app.post('/checkout', asyncHandler(async (req, res) => {  
  // ALL these operations are wrapped automatically
```

```

const user = await User.findById(req.body.userId);
const cart = await Cart.findById(user.cartId);
const products = await Product.find({ _id: { $in: cart.items } });
const total = products.reduce((sum, p) => sum + p.price, 0);
const payment = await processPayment(total);
const order = await Order.create({ user: user.id, products, total, payment });

await sendConfirmationEmail(user.email, order);

res.json({ success: true, order });
}));

```

Pattern 3: With Database Transactions

javascript

```

app.post('/transfer', asyncHandler(async (req, res) => {
  const { fromAccount, toAccount, amount } = req.body;

  // Start transaction
  const session = await mongoose.startSession();
  session.startTransaction();

  try {
    const from = await Account.findById(fromAccount).session(session);
    const to = await Account.findById(toAccount).session(session);

    if (from.balance < amount) {
      throw new Error('Insufficient funds');
    }

    from.balance -= amount;
    to.balance += amount;

    await from.save({ session });
    await to.save({ session });
    await Transaction.create([{ from, to, amount }], { session });

    await session.commitTransaction();
    res.json({ success: true, message: 'Transfer successful' });

  } catch (error) {
    await session.abortTransaction();
  }
}

```



```

        throw error; // asyncHandler catches and passes to error handler
    } finally {
        session.endSession();
    }
}));

```

Creating Reusable asyncHandler Module

asyncHandler.js

javascript

```

// utils/asyncHandler.js

/**
 * Wraps async route handlers to automatically catch errors
 * @param {Function} fn - Async function to wrap
 * @returns {Function} Express middleware function
 */
const asyncHandler = (fn) => {
    return (req, res, next) => {
        Promise.resolve(fn(req, res, next)).catch(next);
    };
};

module.exports = asyncHandler;

```

Using in different files

javascript

```

// routes/users.js

const express = require('express');
const asyncHandler = require('../utils/asyncHandler');
const router = express.Router();

router.get('/', asyncHandler(async (req, res) => {
    const users = await User.find();
    res.json(users);
}));

```

```
router.get('/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) throw new Error('User not found');
  res.json(user);
}));
```

```
module.exports = router;
```

```
// server.js
const userRoutes = require('./routes/users');
app.use('/users', userRoutes);
```

Advanced Wrapper Patterns

Pattern 4: Wrapper with Logging

```
javascript
```

```
const asyncHandlerWithLog = (fn) => {
  return async (req, res, next) => {
    try {
      console.log(`[ASYNC] ${req.method} ${req.url} - Started`);
      await fn(req, res, next);
      console.log(`[ASYNC] ${req.method} ${req.url} - Completed`);
    } catch (error) {
      console.error(`[ASYNC] ${req.method} ${req.url} - Error:`, error.message);
      next(error);
    }
  };
};

app.get('/data', asyncHandlerWithLog(async (req, res) => {
  const data = await fetchData();
  res.json(data);
}));
```

Pattern 5: With Performance Monitoring

```
javascript
```

```
const asyncHandlerWithMetrics = (fn) => {
  return async (req, res, next) => {
    const start = Date.now();
    try {
      await fn(req, res, next);
      const duration = Date.now() - start;
      console.log(`${req.method} ${req.url} took ${duration}ms`);
    } catch (error) {
      const duration = Date.now() - start;
      console.error(`${req.method} ${req.url} failed after ${duration}ms`);
      next(error);
    }
  };
};
```

Pattern 6: Wrapper Array for Middleware

javascript

```
// Multiple middlewares wrapped together
const asyncMiddleware = (middlewares) => {
  return middlewares.map(middleware => {
    return async (req, res, next) => {
      try {
        await middleware(req, res, next);
      } catch (error) {
        next(error);
      }
    };
  });
};

// Usage
const validateUser = async (req, res, next) => {
  const user = await User.findById(req.params.id);
  if (!user) throw new Error('User not found');
  req.user = user;
  next();
};

const checkPermissions = async (req, res, next) => {
  const hasAccess = await checkAccess(req.user, req.body.resource);
```

```

    if (!hasAccess) throw new Error('No permission');
    next();
  };

  app.get('/protected/:id',
    asyncMiddleware([validateUser, checkPermissions]),
    asyncHandler(async (req, res) => {
      res.json({ message: 'Access granted', user: req.user });
    })
  );

```

Real-World Example: Complete E-Commerce API

javascript

```

const express = require('express');
const app = express();
const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

// Product routes
app.get('/api/products', asyncHandler(async (req, res) => {
  const { category, minPrice, maxPrice } = req.query;
  let query = {};

  if (category) query.category = category;
  if (minPrice) query.price = { $gte: minPrice };
  if (maxPrice) query.price = { ...query.price, $lte: maxPrice };

  const products = await Product.find(query);
  res.json({ success: true, count: products.length, data: products });
}));

app.get('/api/products/:id', asyncHandler(async (req, res) => {
  const product = await Product.findById(req.params.id);

  if (!product) {
    const error = new Error('Product not found');
    error.status = 404;
    throw error;
  }

  res.json({ success: true, data: product });
}));

```

```
    }));

// Cart routes
app.post('/api/cart', asyncHandler(async (req, res) => {
    const { userId, productId, quantity } = req.body;

    const product = await Product.findById(productId);
    if (!product) throw new Error('Product not found');

    if (product.stock < quantity) {
        throw new Error(`Only ${product.stock} items available`);
    }

    let cart = await Cart.findOne({ userId });
    if (!cart) {
        cart = new Cart({ userId, items: [] });
    }

    const existingItem = cart.items.find(item => item.productId === productId);
    if (existingItem) {
        existingItem.quantity += quantity;
    } else {
        cart.items.push({ productId, quantity });
    }

    await cart.save();
    res.json({ success: true, data: cart });
}));

// Order routes
app.post('/api/orders', asyncHandler(async (req, res) => {
    const { userId, paymentMethod } = req.body;

    const cart = await Cart.findOne({ userId }).populate('items.productId');
    if (!cart || cart.items.length === 0) {
        throw new Error('Cart is empty');
    }

    let total = 0;
    for (const item of cart.items) {
        total += item.productId.price * item.quantity;
    }

    const order = new Order({
```

```
    userId,
    items: cart.items,
    total,
    paymentMethod,
    status: 'pending'
  });

  await order.save();

  // Clear cart
  cart.items = [];
  await cart.save();

  // Process payment (async)
  const payment = await processPayment(total, paymentMethod);
  order.paymentId = payment.id;
  order.status = 'paid';
  await order.save();

  res.status(201).json({ success: true, data: order });
}));

// Error handler
app.use((err, req, res, next) => {
  const status = err.status || 500;
  const message = err.message || 'Internal Server Error';

  res.status(status).json({
    success: false,
    error: message,
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
});

app.listen(3000);
```

Comparing Try-Catch vs asyncHandler

javascript

// Same functionality, different style

// Using try-catch

```

app.get('/user/:id', async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    res.json(user);
  } catch (error) {
    next(error);
  }
});

// Using asyncHandler
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
}));

// Difference: asyncHandler removes the boilerplate try-catch!

```

Testing asyncHandler

```

javascript

// test-asyncHandler.js
const asyncHandler = require('./asyncHandler');

// Mock Express next function
const mockNext = jest.fn();

// Test successful handler
test('asyncHandler calls function successfully', async () => {
  const mockFn = jest.fn().mockResolvedValue('success');
  const wrapped = asyncHandler(mockFn);
  const req = {}, res = {};

  await wrapped(req, res, mockNext);

  expect(mockFn).toHaveBeenCalledWith(req, res, mockNext);
  expect(mockNext).not.toHaveBeenCalled();
});

// Test error handling
test('asyncHandler catches errors and passes to next', async () => {
  const mockError = new Error('Test error');
  const mockFn = jest.fn().mockRejectedValue(mockError);

```

```

const wrapped = asyncHandler(mockFn);

await wrapped({}, {}, mockNext);

expect(mockNext).toHaveBeenCalledWith(mockError);
});

// Test with custom error
test('asyncHandler handles thrown errors', async () => {
  const mockFn = jest.fn().mockImplementation(() => {
    throw new Error('Thrown error');
  });
  const wrapped = asyncHandler(mockFn);

  await wrapped({}, {}, mockNext);

  expect(mockNext).toHaveBeenCalledWith(expect.any(Error));
});

```

Popular Library: express-async-handler

javascript

```

// Install: npm install express-async-handler

const asyncHandler = require('express-async-handler');

// Use exactly the same way
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
}));

// Works with multiple middlewares
app.post('/user',
  asyncHandler(validateUser), // Middleware
  asyncHandler(async (req, res) => {
    const user = await User.create(req.body);
    res.json(user);
  })
);

```


Best Practices

javascript

```
// ✅ DO: Use asyncHandler for all async routes
app.get('/users', asyncHandler(getUsers));
app.post('/users', asyncHandler(createUser));

// ✅ DO: Throw errors for invalid conditions
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) throw new Error('User not found');
  res.json(user);
}));

// ✅ DO: Set status codes on errors
app.get('/product/:id', asyncHandler(async (req, res) => {
  const product = await Product.findById(req.params.id);
  if (!product) {
    const error = new Error('Product not found');
    error.status = 404;
    throw error;
  }
  res.json(product);
}));

// ❌ DON'T: Mix try-catch with asyncHandler
app.get('/user', asyncHandler(async (req, res, next) => {
  try { // Redundant!
    const user = await User.find();
    res.json(user);
  } catch (error) {
    next(error);
  }
}));

// ❌ DON'T: Wrap sync functions
app.get('/data', asyncHandler((req, res) => {
  const data = syncOperation(); // Not needed for sync
  res.json(data);
}));
```

Quick Reference

Without asyncHandler	With asyncHandler
Need try-catch in every route	No try-catch needed
Must call next(error)	Auto passes to next()
Error handling code repeated	DRY - write once
Easy to forget error handling	Can't forget

Remember: `asyncHandler` is a **time-saver** that removes boilerplate try-catch blocks. One wrapper function, import and use everywhere! 🚀

Mongoose Error

🔍 Read 10 web pages  

Mongoose Errors - Complete Easy Guide

What are Mongoose Errors?

Mongoose errors are **special error types** that occur when database operations fail. They provide detailed information about what went wrong with your data or database connection.

The Most Common Mongoose Errors

Here's a quick reference of the errors you'll encounter most often:

Error Type	When It Happens	Default HTTP Status
ValidationError	Data doesn't match schema rules (required, min, max, etc.)	400
CastError	Wrong data type (e.g., string instead of ObjectId)	400

Error Type	When It Happens	Default HTTP Status
Duplicate Key (11000)	Unique field already exists	409
DocumentNotFoundError	Document doesn't exist when expected	404
MongooseServerSelectionError	Can't connect to MongoDB	503

Error Type 1: ValidationError

What is it?

When you try to save data that **doesn't follow your schema rules** 2 .

Example Schema

javascript

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    minlength: [3, 'Name must be at least 3 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email']
  },
  age: {
    type: Number,
    min: [18, 'Age must be at least 18'],
    max: [120, 'Age cannot exceed 120']
  }
});
```

How ValidationError Looks

javascript

```
// When you try to save invalid data:
try {
  await User.create({ name: 'Jo', email: 'not-an-email', age: 15 });
} catch (error) {
  console.log(error);
  // Error object structure:
  // {
  //   name: 'ValidationError',
  //   message: 'User validation failed',
  //   errors: {
  //     name: { message: 'Name must be at least 3 characters', kind: 'minlength', path: 'name', value: 'Jo' },
  //     email: { message: 'Please enter a valid email', kind: 'regexp', path: 'email', value: 'not-an-email' },
  //     age: { message: 'Age must be at least 18', kind: 'min', path: 'age', value: 15 }
  //   }
  // }
}
```

Handling ValidationError

javascript

```
app.post('/users', async (req, res, next) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json({ success: true, data: user });
  } catch (error) {
    if (error.name === 'ValidationError') {
      // Format validation errors nicely
      const errors = Object.values(error.errors).map(err => ({
        field: err.path,
        message: err.message
      }));
      return res.status(400).json({
        success: false,
        message: 'Validation failed',
        errors: errors
      });
    }
  }
}
```

```
    next(error);  
  }  
});
```

Error Type 2: CastError

What is it?

When you provide a value that **can't be converted to the expected type** ² .

Most Common CastError: Invalid ObjectId

javascript

```
// Schema with ObjectId reference  
const orderSchema = new mongoose.Schema({  
  user: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: 'User',  
    required: true  
  }  
});  
  
// WRONG - Sending invalid ObjectId  
app.get('/orders/:userId', async (req, res, next) => {  
  try {  
    // If userId = 'invalid-id' (not a valid 24-character hex string)  
    const orders = await Order.find({ user: req.params.userId });  
    // Throws CastError: Cast to ObjectId failed for value "invalid-id"  
  } catch (error) {  
    if (error.name === 'CastError') {  
      return res.status(400).json({  
        success: false,  
        message: `Invalid ${error.path}: ${error.value}`  
      });  
    }  
    next(error);  
  }  
});
```

Handling CastError

javascript

```
app.get('/products/:id', async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) {
      return res.status(404).json({ success: false, message: 'Product not found'
    });
  }
  res.json({ success: true, data: product });
} catch (error) {
  // Handle CastError (invalid ObjectId format)
  if (error.name === 'CastError') {
    return res.status(400).json({
      success: false,
      message: 'Invalid ID format',
      error: `The provided ID '${error.value}' is not a valid ObjectId`
    });
  }
  next(error);
}
});
```

Error Type 3: Duplicate Key Error (E11000)

What is it?

When you try to insert a document with a **unique field value that already exists** ³ .

Schema with Unique Fields

javascript

```
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true // ← This creates a unique index
  },
```

```

    username: {
      type: String,
      required: true,
      unique: true // ← Also unique
    }
  });

```

How Duplicate Error Looks

javascript

```

// First user creates successfully
await User.create({ email: 'john@example.com', username: 'john123' });

// Second user with same email
try {
  await User.create({ email: 'john@example.com', username: 'jane456' });
} catch (error) {
  console.log(error);
  // Error object:
  // {
  //   code: 11000, // ← MongoDB error code
  //   keyPattern: { email: 1 },
  //   keyValue: { email: 'john@example.com' },
  //   message: 'E11000 duplicate key error collection: test.users index: email_1 dup k
ey: { email: "john@example.com" }'
  // }
}

```

Handling Duplicate Key Errors

javascript

```

app.post('/users', async (req, res, next) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json({ success: true, data: user });
  } catch (error) {
    // Check for duplicate key error (code 11000)
    if (error.code === 11000) {
      // Extract which field caused the duplicate
    }
  }
}

```

```

const field = Object.keys(error.keyPattern)[0];
const value = error.keyValue[field];

return res.status(409).json({
  success: false,
  message: `Duplicate value error`,
  errors: [{
    field: field,
    message: `${field} '${value}' already exists. Please use a different
    ${field}.`
  }]
});
}
next(error);
}
});

```

⚠ Important Note About Unique

The `unique: true` option creates a **database index**, not a validator. This means:

- `validate()` won't catch duplicate errors
- You must handle the 11000 error code
- Indexes persist even after dropping the database (need to sync indexes) ³

Error Type 4: DocumentNotFoundError

What is it?

When you use `.orFail()` or similar methods and the document doesn't exist ².

javascript

```

// Using orFail() - throws if document not found
app.delete('/users/:id', async (req, res, next) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id).orFail();
    res.json({ success: true, message: 'User deleted' });
  } catch (error) {
    if (error.name === 'DocumentNotFoundError') {

```



```

    return res.status(404).json({
      success: false,
      message: `User with ID ${req.params.id} not found`
    });
  }
  next(error);
}
});

```

Error Type 5: Connection Errors

MongooseServerSelectionError / MongoNetworkError

When Mongoose can't connect to the database 5 .

javascript

```

const mongoose = require('mongoose');

// Connection options to prevent timeout errors
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI, {
      serverSelectionTimeoutMS: 5000, // Time to wait for server selection
      socketTimeoutMS: 45000,        // Time to wait for socket operations
      connectTimeoutMS: 10000,        // Time to wait for initial connection
      maxPoolSize: 10                 // Maximum number of connections
    });
    console.log('MongoDB connected successfully');
  } catch (error) {
    console.error('MongoDB connection error:', error.message);
    // Don't exit - retry connection
    setTimeout(connectDB, 5000);
  }
};

// Handle connection events
mongoose.connection.on('error', (error) => {
  console.error('MongoDB connection error:', error);
});

mongoose.connection.on('disconnected', () => {

```

```
console.log('MongoDB disconnected. Attempting to reconnect...');  
});
```

Complete Error Handler for All Mongoose Errors

javascript

// errorHandler.js

```
const handleMongooseError = (err, req, res, next) => {  
  // 1. Validation Error (400)  
  if (err.name === 'ValidationError') {  
    const errors = Object.values(err.errors).map(e => ({  
      field: e.path,  
      message: e.message,  
      value: e.value  
    }));  
  
    return res.status(400).json({  
      success: false,  
      error: 'Validation Error',  
      message: 'Invalid data provided',  
      details: errors  
    });  
  }  
  
  // 2. Cast Error (400) - Invalid ID format  
  if (err.name === 'CastError') {  
    return res.status(400).json({  
      success: false,  
      error: 'Cast Error',  
      message: `Invalid value for field '${err.path}': '${err.value}'`,  
      details: {  
        field: err.path,  
        value: err.value,  
        expectedType: err.kind  
      }  
    });  
  }  
  
  // 3. Duplicate Key Error (409)  
  if (err.code === 11000) {  
    const field = Object.keys(err.keyPattern)[0];
```

```
const value = err.keyValue[field];

return res.status(409).json({
  success: false,
  error: 'Duplicate Key Error',
  message: `${field} already exists`,
  details: {
    field: field,
    value: value,
    suggestion: `Please use a different ${field}`
  }
});
}

// 4. Document Not Found (404)
if (err.name === 'DocumentNotFoundError') {
  return res.status(404).json({
    success: false,
    error: 'Not Found',
    message: 'The requested document does not exist'
  });
}

// 5. Connection/Server Errors (503)
if (err.name === 'MongooseServerSelectionError' ||
  err.name === 'MongoNetworkError') {
  return res.status(503).json({
    success: false,
    error: 'Database Unavailable',
    message: 'Unable to connect to database. Please try again later.'
  });
}

// 6. Version Error (409) - Concurrent modification
if (err.name === 'VersionError') {
  return res.status(409).json({
    success: false,
    error: 'Version Conflict',
    message: 'Document was modified by another process. Please retry.'
  });
}

// 7. Strict Mode Error (400)
if (err.name === 'StrictModeError') {
```

```

    return res.status(400).json({
      success: false,
      error: 'Strict Mode Violation',
      message: err.message
    });
  }

  // Pass other errors to default handler
  next(err);
};

module.exports = handleMongooseError;

```

Using the Error Handler in Your App

```

javascript

const express = require('express');
const mongoose = require('mongoose');
const handleMongooseError = require('./errorHandler');

const app = express();
app.use(express.json());

// Your routes
app.post('/users', async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json({ success: true, data: user });
  } catch (error) {
    // Mongoose errors will be caught here
    handleMongooseError(error, req, res, (err) => {
      // If not a mongoose error, send generic response
      res.status(500).json({
        success: false,
        error: 'Internal Server Error',
        message: 'Something went wrong'
      });
    });
  }
});

// Or use with asyncHandler

```

```
const asyncHandler = require('express-async-handler');

app.post('/users', asyncHandler(async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json({ success: true, data: user });
}));
// Errors are automatically passed to error handling middleware
```

Special Case: Update Operations Don't Validate by Default

The Problem

Mongoose DOES NOT run validators on `update`, `findOneAndUpdate`, or `updateOne` by default ⁴.

javascript

```
// Schema with required field
const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, required: true }
});

// ❌ This will NOT run validators!
app.put('/products/:id', async (req, res) => {
  // Can send empty body - no validation!
  await Product.findByIdAndUpdate(req.params.id, {});
});
```

The Solution: Add `runValidators: true`

javascript

```
// ✅ This WILL run validators
app.put('/products/:id', async (req, res, next) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      {

```

```

        new: true,           // Return updated document
        runValidators: true, // ← CRITICAL: Run schema validators
        context: 'query'     // Required for some validators
      }
    );

    if (!product) {
      return res.status(404).json({ success: false, message: 'Product not found'
    });
  }

  res.json({ success: true, data: product });
} catch (error) {
  next(error);
}
});

```

Popular Package: express-mongo-error-handler

For a simpler solution, you can use a pre-built package ¹ :

bash

```
npm install express-mongo-error-handler
```

javascript

```

const mongoErrorHandler = require('express-mongo-error-handler');

// Automatically handles all Mongoose/MongoDB errors
app.use(mongoErrorHandler());

// Response format:
// {
//   "success": false,
//   "message": "Duplicate key violation",
//   "errors": [
//     { "field": "email", "message": "Record with field 'email' already exists" }
//   ]
// }

```

Quick Reference: Error Handler by Status Code

javascript

```
// Complete error handling middleware
app.use((err, req, res, next) => {
  // 400 - Bad Request (Validation, Cast)
  if (err.name === 'ValidationError' || err.name === 'CastError') {
    return res.status(400).json({ error: err.message });
  }

  // 404 - Not Found
  if (err.name === 'DocumentNotFoundError') {
    return res.status(404).json({ error: 'Document not found' });
  }

  // 409 - Conflict (Duplicate Key)
  if (err.code === 11000) {
    return res.status(409).json({ error: 'Duplicate value' });
  }

  // 503 - Service Unavailable (Connection errors)
  if (err.name.includes('Connection') || err.name.includes('Network')) {
    return res.status(503).json({ error: 'Database unavailable' });
  }

  // 500 - Everything else
  res.status(500).json({ error: 'Internal server error' });
});
```

Summary

Error	check for	send status
Validation failed	err.name === 'ValidationError'	400
Wrong ID format	err.name === 'CastError'	400
Duplicate unique field	err.code === 11000	409
Document missing	err.name === 'DocumentNotFoundError'	404

Error	check for	send status
Cannot connect	<code>err.name.includes('ServerSelection')</code>	503

Remember: Always add `runValidators: true` for update operations, handle duplicate key errors with code `11000`, and format validation errors to show which fields failed! 🚀

 10 web pages

sumrize all concept from error in easy way to understand

Error Handling in Express - Complete Easy Summary 🎯

The Big Picture (30-Second Read)

Think of error handling like a **safety net** for your app:

- **Without it:** One error crashes everything ✨
- **With it:** Errors are caught, logged, and reported gracefully ✅

javascript

```
// This ONE middleware catches ALL errors
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({ error: err.message || "Something broke!" });
});
```

All Concepts in Simple Terms

1. Error Handling Middleware - The Safety Net

javascript

```
// The 4 parameters = error handler (regular middleware has 3)
app.use((err, req, res, next) => {
  // Always put this LAST after all routes
```



```
res.status(500).json({ error: err.message });
});
```

Key Rule: Place error middleware at the **very end** of all routes and middleware!

2. Default Status & Messages - The Backup Plan

javascript

```
// Use || to provide defaults when info is missing
const status = err.status || 500;    // If no status → 500
const message = err.message || "Error"; // If no message → "Error"

// Why? Because errors might not have all information!
```

Memory trick: `||` means "use this OR that" - left side preferred, right side is backup.

3. Async Errors - The Tricky Ones

The Problem: Express CANNOT catch errors in async functions automatically

javascript

```
// ❌ THIS CRASHES
app.get('/user', async (req, res) => {
  throw new Error('Boom!'); // CRASH!
});

// ✅ THIS WORKS - Use try-catch
app.get('/user', async (req, res, next) => {
  try {
    throw new Error('Boom!');
  } catch(error) {
    next(error); // Pass to error handler
  }
});
```

4. Try-Catch Pattern - The Simple Solution

javascript

```
// The pattern you'll use 90% of the time
app.get('/data', async (req, res, next) => {
  try {
    const data = await databaseQuery();
    res.json(data);
  } catch (error) {
    next(error); // ← Always pass to next!
  }
});
```

Golden Rule: Every async route needs try-catch + next(error)

5. asyncHandler (wrapAsync) - The Clean Solution

Removes the repetitive try-catch code:

javascript

```
// WITHOUT wrapper (repetitive)
app.get('/users', async (req, res, next) => {
  try { const users = await User.find(); res.json(users); }
  catch(e) { next(e); }
});

// WITH wrapper (clean)
const asyncHandler = (fn) => (req, res, next) => fn(req, res, next).catch(next);

app.get('/users', asyncHandler(async (req, res) => {
  const users = await User.find(); // Errors auto-caught!
  res.json(users);
})));
```

6. Custom Error Classes - Smart Errors

javascript

```
// Create meaningful errors
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Use them
throw new AppError('User not found', 404);
// Now error has BOTH message AND status code!
```

Benefit: Errors carry their own status codes - cleaner code!

7. Mongoose Errors - Database Errors

Error Type	When	Status
ValidationError	Missing/Invalid data	400
CastError	Wrong ID format	400
Code 11000	Duplicate unique field	409
DocumentNotFoundError	Document missing	404

javascript

```
// Handle them all
if (err.name === 'ValidationError') return res.status(400).json(...);
if (err.code === 11000) return res.status(409).json(...);
if (err.name === 'CastError') return res.status(400).json(...);
```

 **Complete Working Example (Copy-Paste Ready)**

javascript

```
const express = require('express');
const app = express();

// Helper: asyncHandler wrapper
const asyncHandler = (fn) => (req, res, next) => fn(req, res, next).catch(next);

// Helper: Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Route that might have errors
app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);

  if (!user) {
    throw new AppError('User not found', 404);
  }

  res.json({ success: true, data: user });
})));

// ↑ ALL routes go here ↑

// ↓ Error handlers go here (LAST!)
app.use((err, req, res, next) => {
  // Handle Mongoose ValidationError
  if (err.name === 'ValidationError') {
    return res.status(400).json({ error: err.message });
  }

  // Handle duplicate key error
  if (err.code === 11000) {
    return res.status(409).json({ error: 'Duplicate value' });
  }

  // Handle our custom errors
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({ error: err.message });
  }
});
```

```
}

// Handle everything else (default)
const status = err.status || 500;
const message = err.message || 'Internal Server Error';
res.status(status).json({ error: message });
});

// 404 handler - catches unmatched routes
app.use('*', (req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

app.listen(3000);
```



Quick Decision Guide

Ask yourself: "What kind of error handling do I need?"

text

Is it a simple route?

→ Just use try-catch + next(error)

Are there many async routes?

→ Use asyncHandler wrapper

Need to pass status codes?

→ Use custom error class

Working with MongoDB?

→ Handle Mongoose specific errors

Is my app for production?

→ Add logging and environment-specific responses



Key Takeaways (Remember These!)

1. **Error middleware goes LAST** - After all routes
2. **Always use defaults** - `err.status || 500`
3. **Async needs try-catch** - Express won't catch it
4. **Pass to next()** - `next(error)` triggers error handler
5. **One error handler to rule them all** - Centralized error handling

Quick Reference Card

javascript

```
// 1. Basic error handler (most important!)
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({ error: err.message || "Error" });
});

// 2. Async route with try-catch
app.get('/data', async (req, res, next) => {
  try { res.json(await getData()); }
  catch(e) { next(e); }
});

// 3. asyncHandler (save typing)
const asyncHandler = fn => (req, res, next) => fn(req, res, next).catch(next);

// 4. Custom error
throw { status: 404, message: "Not found" };

// 5. Mongoose errors
if (err.code === 11000) // duplicate
if (err.name === 'ValidationError') // validation
if (err.name === 'CastError') // wrong ID
```

The Golden Rules

text

- Rule 1: Always have an error handler at the end
- Rule 2: Always pass errors to next() in async routes
- Rule 3: Always provide default status and message
- Rule 4: Always put error handler after ALL routes
- Rule 5: One error handler handles ALL errors

Most Common Mistakes (Avoid These!)

javascript

```
// ❌ MISTAKE 1: Error handler in wrong place
app.use(errorHandler); // Too early!
app.get('/route', handler);
```

```
// ✅ CORRECT
app.get('/route', handler);
app.use(errorHandler); // After all routes!
```

```
// ❌ MISTAKE 2: Not using next() in async
app.get('/user', async (req, res) => {
  throw new Error('Boom!'); // CRASH!
});
```

```
// ✅ CORRECT
app.get('/user', async (req, res, next) => {
  try { throw new Error('Boom!'); }
  catch(e) { next(e); }
});
```

```
// ❌ MISTAKE 3: No defaults
app.use((err, req, res, next) => {
  res.status(err.status).json({ error: err.message }); // May be undefined!
});
```

```
// ✅ CORRECT
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({ error: err.message || "Error" });
});
```



Error Status Codes Cheat Sheet

Code	Meaning	When to Use
400	Bad Request	Validation fails, wrong data format
401	Unauthorized	Not logged in
403	Forbidden	Logged in but no permission
404	Not Found	User/product doesn't exist
409	Conflict	Email already exists
429	Too Many Requests	Rate limit hit
500	Server Error	Database down, unexpected error
503	Service Unavailable	External API down



Final Simple Summary

Error handling in 3 sentences:

1. Create ONE middleware with `(err, req, res, next)` at the END of your app
2. Use `||` to provide defaults so you never have undefined values
3. In async routes, wrap in `try-catch` and call `next(error)`

That's it! Everything else is just making errors more specific and user-friendly.



Remember This Pattern Forever

javascript

```
// THE UNIVERSAL ERROR HANDLING PATTERN  
app.use((err, req, res, next) => {
```



```
const status = err.status || 500;  
const message = err.message || 'Server Error';  
res.status(status).json({ error: message });  
});
```

Master this one pattern, and you've mastered 80% of error handling! 🎯